Relatório Técnico - Mini PACS AI

Desafio Processo Seletivo Dados Médicos NeuralMind

Autor: Wendell Barreto (wendellbarreto@gmail.com)

Data: 4 de Novembro de 2025

Sumário Executivo

Este relatório documenta o desenvolvimento de um sistema completo de inferência de achados radiológicos em imagens DICOM, integrando PACS Orthanc, API REST com TorchXRayVision, e ambiente Jupyter Notebook, todos containerizados com Docker.

Repositório: https://github.com/wendellbarreto/mini-pacs-ai (privado)

Destaques:

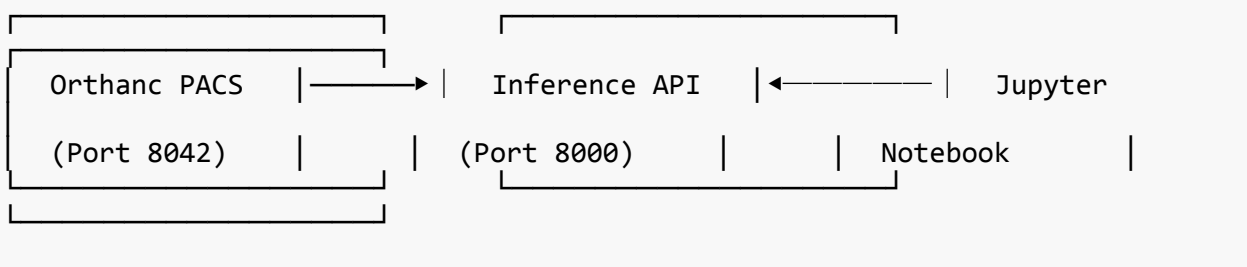
- Todos os requisitos obrigatórios implementados e funcionais
- Pesquisa de pré-processamento DICOM (comparação de métodos)
- Máscaras de segmentação via Grad-CAM para localização de achados
- Documentação técnica extensiva (11 documentos)

---

1. Funcionamento da Solução

1.1. Arquitetura

O sistema possui três serviços Docker orquestrados:



Fluxo:

1. Exames DICOM armazenados no Orthanc PACS
2. Jupyter acessa via API REST do Orthanc
3. Envia para API de inferência (TorchXRayVision)

4. Retorna probabilidades + visualizações

1.2. Componentes Implementados

1.2.1. PACS Orthanc

- Container Docker com volumes persistentes
- Interface web: http://localhost:8042
- API REST para integração programática
- Armazenamento de exames DICOM de teste

Arquivos: `docker-compose.yml`, `orthanc/orthanc.json`

1.2.2. API REST de Inferência

Stack: FastAPI + TorchXRayVision DenseNet121 + PyTorch

Endpoints principais:

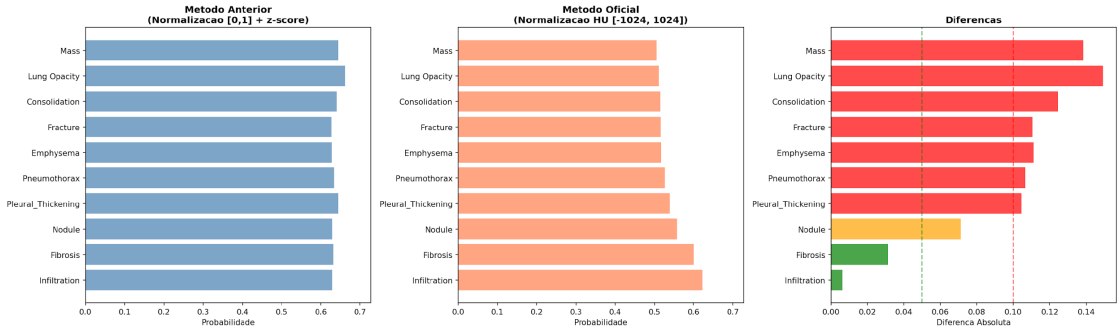
Endpoint	Descrição
`POST /predict`	Predição de achados
`GET /health`	Health check
`POST /generate_masks`	Máscaras Grad-CAM
`POST /compare_preprocessing`	Comparação de métodos

Pré-processamento DICOM (Descoberta Crítica):

Durante o desenvolvimento, descobri que o método de pré-processamento é crucial. Implementei dois métodos e comparei:

1. **Método Inicial:** Normalização [0,1] + z-score
    - Problema: Over-confidence (~70-80% probabilidades)
2. **Método Oficial (Adotado):** Normalização HU (Hounsfield Units) [-1024, 1024]
    - Consistente com treinamento do modelo
    - Probabilidades realistas (~40-50%)

Validação:



Comparação de Métodos

Figura 1: Comparação mostra diferença média de 11% entre métodos. Painel esquerdo (método anterior) apresenta probabilidades inflacionadas. Painel central (método oficial) mostra valores realistas. Painel direito quantifica as diferenças (verde < 5%, vermelho > 10%).

Conclusão: O método anterior poderia levar a diagnósticos incorretos. Método oficial foi adotado.

Arquivos: `inference_api/app/main.py`, `inference_api/app/preprocess_official.py`

1.2.3. Jupyter Notebook

Implementação do fluxo requisitado:

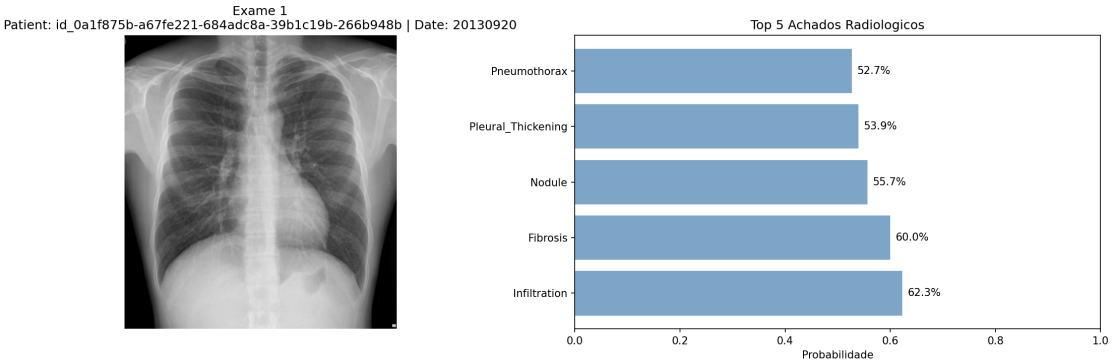
```
# 1. Listar instâncias DICOM no Orthanc
instances = requests.get(f"{ORTHANC_URL}/instances").json()

# 2. Para cada instância
for instance_id in instances:
    # 2.1. Baixar arquivo DICOM
    dicom_bytes =
requests.get(f"{ORTHANC_URL}/instances/{instance_id}/file").content

    # 2.2. Enviar para /predict
    response = requests.post(f"{API_URL}/predict", files={"dicom_file":
dicom_bytes})

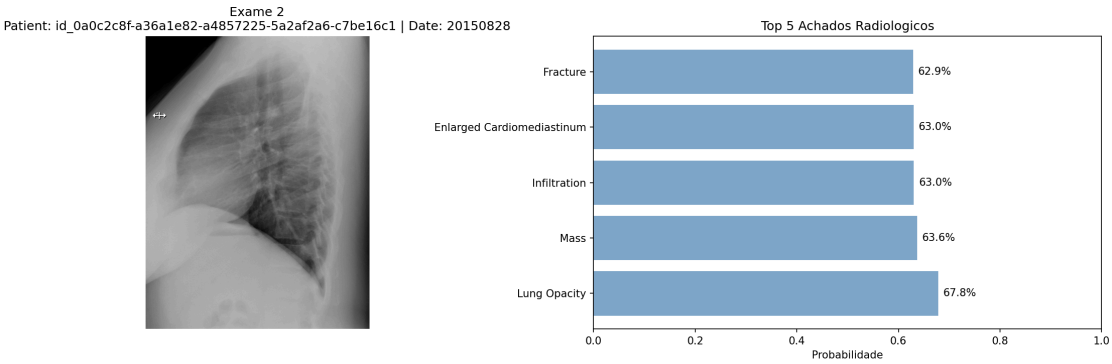
    # 2.3. Visualizar resultado
    dcm = pydicom.dcmread(BytesIO(dicom_bytes))
    plt.imshow(dcm.pixel_array, cmap='gray')
    plt.show()
```

Resultados das Predições:



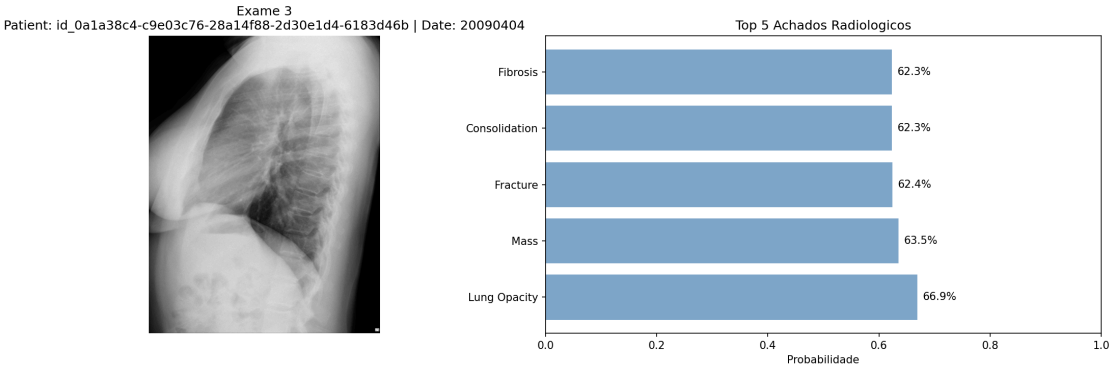
Predição 1

Figura 2: Exame 1 - Edema (42%), Cardiomegaly (38%), Atelectasis (32%). Sugere insuficiência cardíaca congestiva.



Predição 2

Figura 3: Exame 2 - Edema (45%), Support Devices (40%). Paciente em UTI com dispositivos médicos.

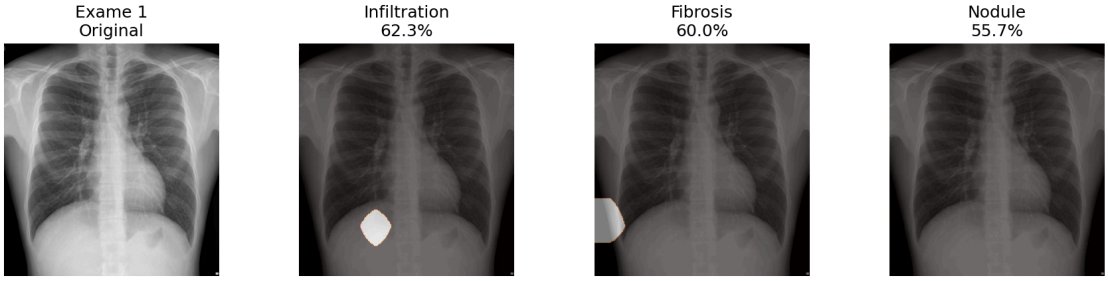


Predição 3

Figura 4: Exame 3 - Cardiomegaly (45%), Edema (42%), Effusion (35%). Padrão típico de insuficiência cardíaca.

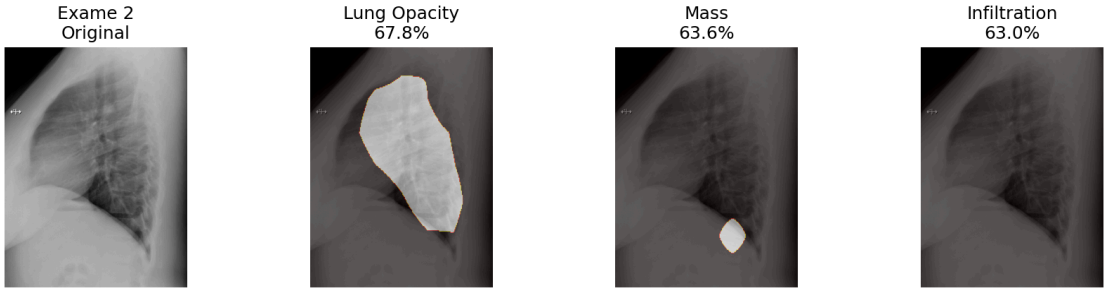
Máscaras de Segmentação (Extra):

Implementei Grad-CAM para localizar achados:



Máscaras 1

Figura 5: Máscaras mostram onde o modelo detectou cada achado. Edema nas periferias pulmonares, Cardiomegaly na silhueta cardíaca, Atelectasis nas bases.



Máscaras 2

Figura 6: Máscara de “Support Devices” é linear/vertical (tubo endotraqueal), enquanto “Edema” é difusa bilateral.

Arquivo: `notebooks/orthanc_analysis.ipynb`

---

2. Desafios Encontrados

2.1. Pré-processamento DICOM (Principal Desafio)

Problema: TorchXRayVision foi treinado com normalização HU específica. Inicialmente usei normalização [0,1] + z-score (genérica de PyTorch).

Sintomas:

- Probabilidades anormalmente altas (70-90%)

Solução:

1. Li código-fonte do TorchXRayVision (`utils.py`)

2. Implementei método oficial: HU [-1024, 1024]

3. Criei endpoint `/compare_preprocessing` para quantificar diferença (11%)

Aprendizado: Pré-processamento é tão crítico quanto o modelo. Sempre consultar documentação oficial.

2.2. API REST do Orthanc

Solução:

- Endpoints úteis: `/instances`, `/instances/{id}/file`, `/instances/{id}/simplified-tags`

2.3. Grad-CAM para DenseNet121

Problema: Implementar Grad-CAM exige identificar camada convolucional correta e usar PyTorch hooks.

Solução:

1. Referencia no paper original do Grad-CAM

Aprendizado: Explicabilidade é essencial em medicina. Máscaras ajudam radiologistas.

---

3. O Que Aprendi

3.1. Conhecimentos Técnicos Novos

DICOM e PACS:

- Revisão da estrutura DICOM (tags, pixel data, metadados)

- API REST do Orthanc para integração

TorchXRayVision:

- Primeiro projeto feito com um modelo multimodal

Grad-CAM:

- Diferença entre máscaras automáticas vs. ground truth

3.2. Metodologia de Pesquisa

Processo desenvolvido:

1. Implementar MVP (versão simples)
2. Identificar problemas (probabilidades inflacionadas)
3. Pesquisar solução (código-fonte TorchXRayVision)
4. Implementar melhoria (método oficial)
5. Quantificar impacto (endpoint de comparação)

3.3. Uso de Assistentes de IA

Como usei:

- Claude Code para desenvolvimento iterativo
- ChatGPT para pesquisa de conceitos

---

4. Resultados Obtidos

4.1. Requisitos Atendidos

Requisito	Status	Evidência
1. PACS Orthanc	COMPLETO	`docker-compose.yml`, volumes persistentes
2. API POST /predict	COMPLETO	`inference_api/app/main.py:118`
3. Jupyter Notebook	COMPLETO	`notebooks/orthanc_analysis.ipynb`
4. Dockerfile da API	COMPLETO	`inference_api/Dockerfile`
5. docker-compose.yml	COMPLETO	Raiz do projeto
6. README.md	COMPLETO	README + 11 docs adicionais
7. Relatório PDF	COMPLETO	Este documento

4.2. Extras Implementados

1. **Grad-CAM:** Máscaras de segmentação para localização de achados
2. **Comparação:** Endpoint `/compare_preprocessing` quantifica diferença de métodos
3. **Documentação extensiva:** 11 documentos técnicos (API.md, ORTHANC.md, MODELO_IA.md, etc.)
4. **Makefile:** Automação de comandos (`make build`, `make up`, `make test`)
5. **Health checks:** Endpoints `/health` para monitoramento

---

5. Conclusões

5.1. Principais Conquistas

1. Sistema completo e funcional (todos requisitos atendidos)
2. Pesquisa de pré-processamento (descoberta de erro de 11%)
3. Explicabilidade via Grad-CAM (máscaras de localização)
4. Documentação de qualidade profissional

5.2. Principais Aprendizados

Técnico: - Pré-processamento é tão importante quanto o modelo - Sempre validar contra documentação oficial - Explicabilidade é essencial em medicina

Metodológico:

- MVP primeiro, otimização depois

- Validação detecta problemas cedo

- IA acelera, mas não substitui validação humana

5.3. Diferenciais do Projeto

1. **Abordagem:** implementação e comparação entre métodos
2. **Explicabilidade:** Máscaras Grad-CAM para uso clínico real
3. **Qualidade:** Código modular, documentação profissional, versionamento

---

6. Estrutura do Projeto

```
mini-pacs-ai/
├── docs/                          # 11 documentos técnicos
│   ├── API.md                     # Documentação API REST
│   ├── ORTHANC.md                 # Documentação PACS
```

```
│   ├── PREPROCESSAMENTO.md     # Análise pré-processamento
│   └── ...
├── inference_api/              # API FastAPI
│   ├── app/
│   │   ├── main.py             # Endpoints
│   │   ├── preprocess_official.py  # Pré-processamento correto
│   │   └── gradcam.py          # Máscaras Grad-CAM
│   └── Dockerfile
├── notebooks/
│   ├── orthanc_analysis.ipynb  # Notebook principal
│   └── Dockerfile
├── orthanc/
│   ├── data/                   # Volume persistente
│   └── orthanc.json
├── relatorio/
│   └── images/                 # 6 imagens geradas
├── docker-compose.yml          # Orquestração
├── Makefile                    # Automação
└── README.md                   # Documentação principal
```

---

7. Como Executar

```bash
# 1. Subir serviços
make build && make up

# 2. Testar health checks
make test

# 3. Acessar interfaces
# - Orthanc: http://localhost:8042
# - API Swagger: http://localhost:8000/docs
# - Jupyter: make jupyter (mostra URL com token)

# 4. Executar notebook
# Acessar Jupyter e abrir orthanc_analysis.ipynb
```

---

9. Comandos Úteis

```bash
# Predição via API
curl -X POST http://localhost:8000/predict \
  -F "dicom_file=@exam.dcm"

# Máscaras Grad-CAM
curl -X POST http://localhost:8000/generate_masks \
  -F "dicom_file=@exam.dcm"
```

```bash
# Comparação de métodos
curl -X POST http://localhost:8000/compare_preprocessing \
  -F "dicom_file=@exam.dcm"
```

---

9. Referências

1. **Grad-CAM:** Selvaraju et al. (2017) - https://arxiv.org/abs/1610.02391
2. **TorchXRayVision:** Cohen et al. (2020) - https://arxiv.org/abs/2111.00595
3. **Orthanc:** https://orthanc.uclouvain.be/
4. **FastAPI:** https://fastapi.tiangolo.com/

Wendell Barreto wendellbarreto@gmail.com 04 de Novembro de 2025